

変数と関数の定義

環境に名前を結び付ける

情報システム学科 3年・選択・2単位

今日の到達目標

- 環境を使って名前を検索できる
- `define` と `lambda` の役割を区別できる
- クロージャの環境と再帰呼出しを説明できる

前回との接続

数値と演算子だけでも計算はできた。

名前を付けると、計算の意図と再利用する部分を表せる。

評価器に「名前を探す場所」を渡す。

今日の中心

当日は名前の定義、単純な関数、`if` を順に確認する。

クロージャの環境寿命と再帰の復帰は、図で理由を説明する。

未完の実装は継続する。授業内の進め方であり、提出課題の仕様は従来どおり。

変数の実演

```
(define width 5)  
(define height 3)  
(* width height)
```

同じREPLで順に入力すると最後の値は 15。

例を改行して示すが、教材REPLへは1行1式で入力する。

環境の表現

名前	値
width	5
height	3
+	組込み手続き

Pythonホストでは辞書と外側への参照で表せる。

名前の評価

数値の評価は数値自身を返す。

記号の評価は環境を検索する。

`width` が見つからない場合は、未定義名のエラーを返す。

defineの評価

```
(define width (+ 2 3))
```

右辺を現在の環境で評価し、得た値を `width` に登録する。

`width` 自体を先に評価すると、初めての定義が失敗する。

特殊形式

`define` は引数をすべて先に評価できない。

評価の仕方を個別に決める構文を特殊形式と呼ぶ。

通常の関数呼出しと分岐を分ける。

lambdaの実演

```
(define square  
  (lambda (x) (* x x)))  
(square 6)
```

lambda は関数の値を作る。最後の値は 36。

関数の保存内容

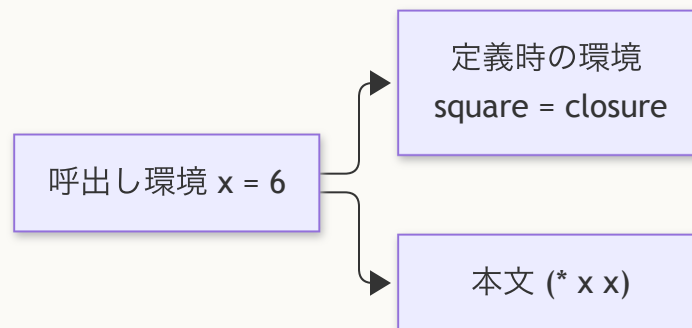
```
@dataclass
class Closure:
    parameters: list
    body: list
    environment: Env
```

完成ホストの定義を抜粋。仮引数、本文の列、定義時環境を保持する。呼び出すまで本文を評価しない。

呼出しの順序

1. 関数の式を評価する
2. 引数の式を評価する
3. 仮引数を新しい環境へ束縛する
4. 本文を評価する

呼出しの環境



新しい環境の外側は、関数が保存した環境。

引数個数

(square 6 7) は仮引数と実引数の個数が合わない。

余分な引数を黙って捨てると、誤った呼出しを見逃す。

束縛する前に個数を確認する。

レキシカルスコープ

名前の意味は、関数を定義した場所の環境に従う。

呼び出した場所の同名変数へ勝手に切り替わらない。

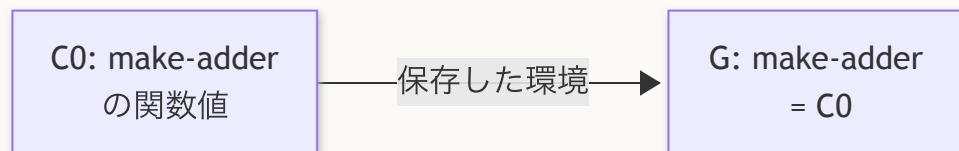
次の例で、保存する環境を追う。

クロージャの例

```
(define make-adder  
  (lambda (x) (lambda (y) (+ x y))))  
(define add5 (make-adder 5))  
(add5 3)
```

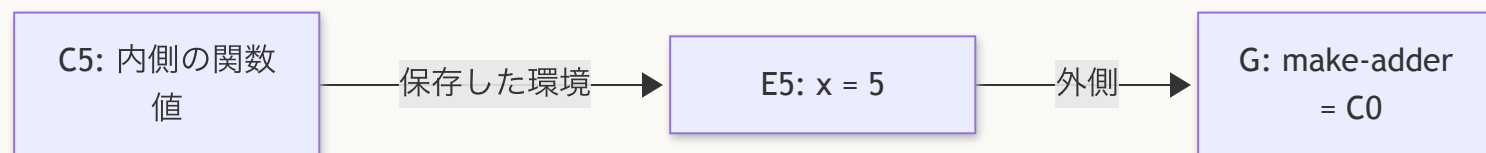
結果 8 に至る4状態を追う。REPLでは各定義を1行にして入力する。図のG・E5・E3とC0・C5は説明用のラベル。

1 make-adderの定義



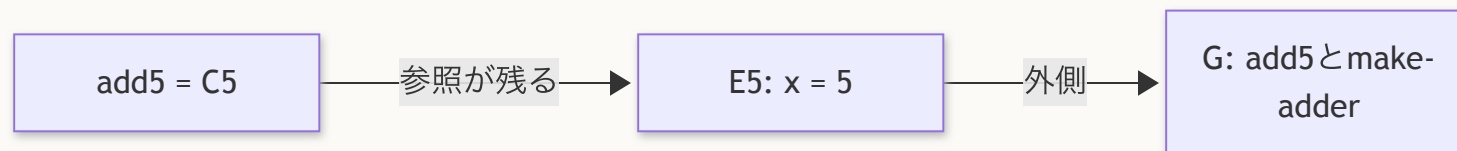
仮引数は `(x)`、本文は内側の `lambda`。定義時はまだ `x = 5` も `y = 3` も作らない。

2 外側の呼出し



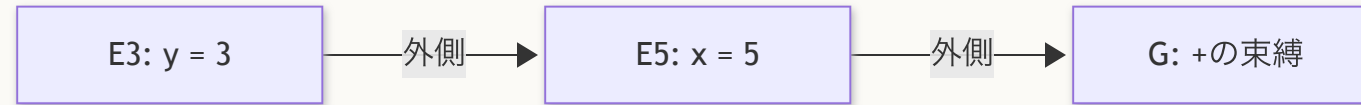
(make-adder 5) はE5で内側の `lambda` を評価する。 `(+ x y)` を計算せず、E5を保存したC5を返す。

3 返された関数



外側の呼出しは終了し、Gの `add5` にC5を登録する。C5からE5へ参照が残るので、`x = 5` を後から使える。

4 add5の呼出し



(add5 3) は保存環境E5を外側にしたE3を作る。y はE3、x はE5、+ はGで見つかり、結果は 8。

同名の引数

```
(define x 100)
(define f (lambda (x) (+ x 1)))
(f 4)
x
```

結果は順に 5 と 100。内側の束縛が外側を隠す。

条件式

```
(if (= 2 2) 10 20)  
(if #f (/ 1 0) 7)
```

選んだ枝だけ評価する。結果は 10 と 7。

ifの評価

条件を評価し、真偽に応じて枝を1つ選ぶ。

両方の枝を通常の引数と同じように評価すると、不要なゼロ除算が起こる。

教材方言の偽値は `#f`。

quoteの評価

```
(+ 1 2)  
(quote (+ 1 2))
```

上は計算、下は式をデータとして返す。

記号 `+` を環境から検索しない。

コードとデータ

リストをプログラムにもデータにも使う。

`quote` で評価を止めれば、別の評価器へ式を渡せる。

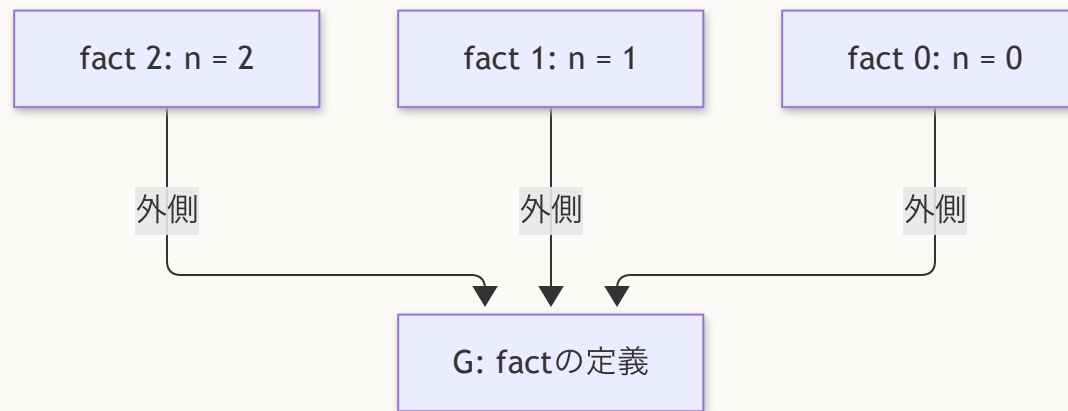
第12回のLisp製評価器へつながる。

再帰関数

```
(define fact
  (lambda (n)
    (if (= n 0)
        1
        (* n (fact (- n 1)))))
(fact 4)
```

停止条件に到達する入力を選ぶ。結果は 24。

再帰の下降



呼出しは 2、1、0 の順。各 n の環境は別。同じGで `fact` を探し、 $n = 0$ から戻る。

再帰の復帰

戻る呼出し	残っていた計算	戻り値
fact 0	停止条件の 1	1
fact 1	(* 1 1)	1
fact 2	(* 2 1)	2

fact 2 は、fact 1 が返るまで乗算を待つ。呼出しの順と、値が戻る順を区別する。

演習1 名前

当日は `define` と未定義名の診断から確認する。

定義前後で同じ名前を評価し、環境の変化を記録する。

動く地点を確かめてから、単純な関数へ進む。

演習2 関数

`square` の呼出しと引数個数の反例を確かめる。

`make-adder` の4状態を説明し、保存する環境を示す。

自作で未完なら完成参照で理解を確認し、実装差分を残す。

演習3 分岐と再帰

`if` が選ばない枝を評価しないことを確かめる。

`fact 2` の下降と復帰を説明し、既存の `fact 4 = 24` も確認する。

未完の機能は、この記録を使って継続して実装する。

確認問い

`lambda` の本文を定義時に評価すると何が起きるか。

クロージャが呼出し元の環境を保存する実装では、どんな反例が作れるか。

解答 lambdaの評価時点

```
(define square (lambda (x) (* x x)))  
(square 6)
```

大域にも `x` がない新しい環境で考える。定義時に本文を評価すると、仮引数 `x` は未束縛で診断になる。

本文は呼出し時に評価する。`x = 6` を束縛した結果は `36`。

解答 保存先の反例

```
(define x 100)
(define make-adder
  (lambda (x) (lambda (y) (+ x y))))
(define add5 (make-adder 5))
(add5 3)
```

REPLでは、各 `define` 全体を1行にまとめて入力する。

正しい保存先はE5の `x = 5`。 `y = 3` と加えて `8`。

誤って内側の関数がGを保存すると、大域の `x = 100` を使い `103`。呼出しから戻っても、定義時環境への参照を保持する。

AI利用の確認

AIの提案した環境検索を、自分の図と比べる。

定義時と呼出し時の環境を混同していないか確かめる。

採用した案と、自分で実行した反例を残す。

まとめと次回

名前の意味を環境で決め、関数は定義時環境を保存した。

特殊形式は評価の順序を制御する。

次回は複数の式をファイルから実行する。

参考

- 本教材: `examples/10-lisp-functions/`
- 補足観察: `learning_trace.py` `closure` / `fact`
- 公開教員補足・演習
- The Scheme Programming Language

教材方言との違いは仕様で確認する。